题目二 查询执行

测试点1:尝试建表

测试示例:

```
create table t1(id int,name char(4));

show tables;

create table t2(id int);

show tables;

drop table t1;

show tables;

drop table t2;

show tables;
```

期待输出:

```
| Tables |
| t1 |
| Tables |
| t1 |
| t2 |
| Tables |
| t2 |
| Tables |
```

测试点2:单表插入与条件查询

测试示例:

```
create table grade (name char(20),id int,score float);

insert into grade values ('Data Structure', 1, 90.5);

insert into grade values ('Data Structure', 2, 95.0);

insert into grade values ('Calculus', 2, 92.0);

insert into grade values ('Calculus', 1, 88.5);
```

```
select * from grade;

select score,name,id from grade where score > 90;

select id from grade where name = 'Data Structure';

select name from grade where id = 2 and score > 90;
```

期待输出：

```
| name | id | score |
| Data Structure | 1 | 90.500000 |
| Data Structure | 2 | 95.000000 |
| Calculus | 2 | 92.000000 |
| Calculus | 1 | 88.500000 |
| score | name | id |
| 90.500000 | Data Structure | 1 |
| 95.000000 | Data Structure | 2 |
| 92.000000 | Calculus | 2 |
| id |
| 1 |
| 2 |
| name |
| Data Structure |
| Calculus |
```

测试点3：单表更新与条件查询

测试示例：

```
create table grade (name char(20),id int,score float);

insert into grade values ('Data Structure', 1, 90.5);

insert into grade values ('Data Structure', 2, 95.0);

insert into grade values ('Calculus', 2, 92.0);

insert into grade values ('Calculus', 1, 88.5);

select * from grade;

update grade set score = 90 where name = 'Calculus' ;

select * from grade;

update grade set name = 'Error name' where name > 'A';

select * from grade;
```

```
update grade set name = 'Error' ,id = -1,score = 0 where name = 'Error name' and
score >= 90;

select * from grade;
```

期待输出：

```
| name | id | score |
| Data Structure | 1 | 90.500000 |
| Data Structure | 2 | 95.000000 |
| Calculus | 2 | 92.000000 |
| Calculus | 1 | 88.500000 |
| name | id | score |
| Data Structure | 1 | 90.500000 |
| Data Structure | 2 | 95.000000 |
| Calculus | 2 | 90.000000 |
| Calculus | 1 | 90.000000 |
| name | id | score |
| Error name | 1 | 90.500000 |
| Error name | 2 | 95.000000 |
| Error name | 2 | 90.000000 |
| Error name | 1 | 90.000000 |
| name | id | score |
| Error | -1 | 0.000000 |
| Error | -1 | 0.000000 |
| Error | -1 | 0.000000 |
| Error | -1 | 0.000000 |
```

题目三 唯一索引

测试点1：创建、删除、展示索引

测试示例：

```
create table warehouse (id int, name char(8));
create index warehouse (id);
show index from warehouse;
create index warehouse (id,name);
show index from warehouse;
drop index warehouse (id);
drop index warehouse (id,name);
show index from warehouse;
```

期待输出：

```
| warehouse | unique | (id) |
| warehouse | unique | (id) |
| warehouse | unique | (id,name) |
```

测试点2：索引查询

测试示例：

```
create table warehouse (w_id int, name char(8));
insert into warehouse values (10, 'qweruiop');
insert into warehouse values (534, 'asdfhjkl');
insert into warehouse values (100,'qwerghjk');
insert into warehouse values (500,'bgtyhnmj');
create index warehouse(w_id);
select * from warehouse where w_id = 10;
select * from warehouse where w_id < 534 and w_id > 100;
drop index warehouse(w_id);
create index warehouse(name);
select * from warehouse where name = 'qweruiop';
select * from warehouse where name > 'qwerghjk';
select * from warehouse where name > 'aszdefgh' and name < 'qweraaaa';
drop index warehouse(name);
create index warehouse(w_id,name);
select * from warehouse where w_id = 100 and name = 'qwerghjk';
select * from warehouse where w_id < 600 and name > 'bztyhnmj';
```

期待输出：

```
| w_id | name |
| 10 | qweruiop |
| w_id | name |
| 500 | bgtyhnmj |
| w_id | name |
| 10 | qweruiop |
| w_id | name |
| 10 | qweruiop |
| w_id | name |
| 500 | bgtyhnmj |
| w_id | name |
| 100 | qwerghjk |
| w_id | name |
| 10 | qweruiop |
| 100 | qwerghjk |
```

测试点3：索引维护

测试示例：

```
create table warehouse (w_id int, name char(8));
insert into warehouse values (10 , 'qweruiop');
insert into warehouse values (534, 'asdfhjkl');
select * from warehouse where w_id = 10;
select * from warehouse where w_id < 534 and w_id > 100;
create index warehouse(w_id);
insert into warehouse values (500, 'lastdanc');
insert into warehouse values (10, 'uiopqwer');
update warehouse set w_id = 507 where w_id = 534;
select * from warehouse where w_id = 10;
select * from warehouse where w_id < 534 and w_id > 100;
drop index warehouse(w_id);
create index warehouse(w_id,name);
insert into warehouse values(10,'qqqqoooo');
insert into warehouse values(500,'lastdanc');
update warehouse set w_id = 10, name = 'qqqqoooo' where w_id = 507 and name =
'asdfhjkl';
select * from warehouse;
```

期待输出：

```
| w_id | name |
| 10 | qweruiop |
| w_id | name |
failure
| w_id | name |
| 10 | qweruiop |
| w_id | name |
| 500 | lastdanc |
| 507 | asdfhjkl |
failure
failure
| w_id | name |
| 10 | qqqqoooo |
| 10 | qweruiop |
| 500 | lastdanc |
| 507 | asdfhjkl |
```

测试点4：是否真正使用单列索引来进行查询

测试示例：

```
create table warehouse (w_id int,name char(8));
insert into warehouse values(1,'12345678');
insert into warehouse values(2,'12345278');
...
insert into warehouse values(2999,'13345678');
insert into warehouse values(3000,'34245418');
```

```
-- 后台计时开始
select * from warehouse where w_id = 1;
select * from warehouse where w_id = 2;
...
select * from warehouse where w_id = 2999;
select * from warehouse where w_id = 3000;
-- 后台计时结束，对比期望输出

create index warehouse(w_id);

-- 后台计时开始
select * from warehouse where w_id = 1;
select * from warehouse where w_id = 2;
...
select * from warehouse where w_id = 2999;
select * from warehouse where w_id = 3000;
-- 后台计时结束，对比期望输出
-- 对比两次查询的耗时
```

期待输出:

```
| w_id | name |
| 1 | 12345678 |
| w_id | name |
| 2 | 12345278 |
...
| w_id | name |
| 2999 | 13345678 |
| w_id | name |
| 3000 | 34245418 |
```

测试点5：是否真正使用多列索引来进行查询

测试示例:

```
create table warehouse (w_id int,name char(8),flo float);
insert into warehouse values(1,'12345678',1024.5);
insert into warehouse values(2,'12345278',512.5);
...
insert into warehouse values(2999,'13345678',256.5);
insert into warehouse values(3000,'34245418',128.5);

-- 后台计时开始
select * from warehouse where w_id = 1 and flo = 1024.500000;
select * from warehouse where w_id = 2 and flo = 512.500000;
...
select * from warehouse where w_id = 2999 and flo = 256.500000;
select * from warehouse where w_id = 3000 and flo = 128.500000;
```

```
-- 后台计时结束，对比期望输出

create index warehouse(w_id,flo);

-- 后台计时开始
select * from warehouse where w_id = 1 and flo = 1024.500000;
select * from warehouse where w_id = 2 and flo = 512.500000;
...
select * from warehouse where w_id = 2999 and flo = 256.500000;
select * from warehouse where w_id = 3000 and flo = 128.500000;
-- 后台计时结束，对比期望输出
-- 对比两次查询的耗时
```

期待输出:

```
| w_id | name | flo |
| 1 | 12345678 | 1024.500000 |
| w_id | name | flo |
| 2 | 12345278 | 512.500000 |
...
| w_id | name | flo |
| 2999 | 13345678 | 256.500000 |
| w_id | name | flo |
| 3000 | 34245418 | 128.500000 |
```

测试文件说明

- storage_test3：创建、删除、展示索引。
- storage_test4：索引查询。
- storage_test5：索引维护。需要保证建有索引的表中，不存在任意两个元组具有相同的索引属性值，当要插入或者修改的元组违背了唯一索引的要求时向output.txt输入failure。在测试文件中，不存在以下情况：在某张表上建立索引前，表中已经存在两个元组具有相同的索引属性值。
- judge_whether_use_index_on_single_attribute：创建表并插入数据后，进行大量单列查询，记录耗时time_a，在某列创建索引，再次进行大量单列查询，记录耗时time_b。若time_b / time_a * 100% <= 70%，视为在单列查询时使用了索引。若判断为没有使用索引，则测试点2和测试点3零分。
- judge_whether_use_index_on_multiple_attributes：创建表并插入数据后，进行大量多列查询，记录耗时time_a，创建多列索引，再次进行大量多列查询，记录耗时time_b。若time_b / time_a * 100% <= 70%，视为在多列查询时使用了索引。若判断为没有使用索引，则测试点2和测试点3零分。

题目四 查询优化与执行

测试点1： 单表查询示例

数据准备

```
CREATE TABLE t (a int, b int);
INSERT INTO t VALUES (1, 5);
INSERT INTO t VALUES (2, 8);
INSERT INTO t VALUES (3, 12);
INSERT INTO t VALUES (4, 6);
INSERT INTO t VALUES (5, 20);
```

执行查询

```
SELECT a, b FROM t WHERE a > 1 AND b < 10;
EXPLAIN ANALYZE SELECT a, b FROM t WHERE a > 1 AND b < 10;
```

SELECT 结果:

```
| a | b |
| 2 | 8 |
| 4 | 6 |
```

EXPLAIN ANALYZE 期望输出:

```
Project(columns=[t.a, t.b],  rows=2)
    Filter(condition=[t.a>1, t.b<10],  rows=2)
        Scan(table=t,  type=SeqScan,  rows=5)
```

说明:

- `Scan rows=5`：扫描 t 表全部 5 行。
- `Filter rows=2`：满足 `a > 1 AND b < 10` 的记录共 2 行。
- `Project rows=2`：投影不改变行数，最终输出 2 行。

测试点2：选择运算下推

数据准备

```
CREATE TABLE orders (
    order_id int,
    customer_id int,
    order_date char(40),
    total_amount float
);
CREATE TABLE customers (
    customer_id int,
```

```
    name char(50),
    email char(100),
    address char(200)
);

INSERT INTO customers VALUES (1, 'Alice', 'alice@example.com', 'A Street');
INSERT INTO customers VALUES (2, 'Bob', 'bob@example.com', 'B Street');
INSERT INTO customers VALUES (3, 'Carol', 'carol@example.com', 'C Street');
INSERT INTO orders VALUES (101, 1, '2025-01-01', 500.0);
INSERT INTO orders VALUES (102, 1, '2025-01-02', 1200.0);
INSERT INTO orders VALUES (103, 2, '2025-01-03', 900.0);
INSERT INTO orders VALUES (104, 2, '2025-01-04', 1500.0);
INSERT INTO orders VALUES (105, 3, '2025-01-05', 700.0);
```

执行查询

```
SELECT * FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    WHERE o.total_amount > 1000;
EXPLAIN ANALYZE SELECT * FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    WHERE o.total_amount > 1000;
```

SELECT 结果:

```
| customer_id | name | email | address | order_id | customer_id | order_date |
total_amount |
| 1 | Alice | alice@example.com | A Street | 102 | 1 | 2025-01-02 | 1200.000000 |
| 2 | Bob | bob@example.com | B Street | 104 | 2 | 2025-01-04 | 1500.000000 |
```

EXPLAIN ANALYZE 期望输出:

```
Project(columns=[*],  rows=2)
    Join(tables=[customers, orders],  condition=[c.customer_id=o.customer_id],
rows=2)
        Scan(table=customers,  type=SeqScan,  rows=3)
        Filter(condition=[o.total_amount>1000],  rows=6)
            Scan(table=orders,  type=SeqScan,  rows=15)
```

说明:

- `Filter` 节点被下推到 orders 表扫描之上，先过滤出 2 条订单。
- `Scan(table=orders) rows=5`：orders 表单次扫描 5 行。
- `Scan(table=customers) rows=6`：在 NLJ 中 customers 作为右侧内表，被左侧 2 条 orders 过滤结果重复扫描，共访问 `2 × 3 = 6` 行。

- `Join rows=2`：过滤后的两条订单分别匹配到一个客户，共输出 2 行。

测试点3：投影下推

数据准备

```
CREATE TABLE orders (
    order_id int,
    customer_id int,
    order_date char(40),
    total_amount float
);

CREATE TABLE customers (
    customer_id int,
    name char(50),
    email char(100),
    address char(200)
);

INSERT INTO customers VALUES (1, 'Alice', 'alice@example.com', 'A Street');
INSERT INTO customers VALUES (2, 'Bob', 'bob@example.com', 'B Street');
INSERT INTO customers VALUES (3, 'Carol', 'carol@example.com', 'C Street');
INSERT INTO orders VALUES (101, 1, '2025-01-01', 500.0);
INSERT INTO orders VALUES (102, 1, '2025-01-02', 1200.0);
INSERT INTO orders VALUES (103, 2, '2025-01-03', 900.0);
INSERT INTO orders VALUES (104, 2, '2025-01-04', 1500.0);
INSERT INTO orders VALUES (105, 3, '2025-01-05', 700.0);
```

执行查询

```
SELECT c.name, o.order_id
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id;

EXPLAIN ANALYZE SELECT c.name, o.order_id
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id;
```

SELECT 结果:

```
| name | order_id |
| Alice | 101 |
| Alice | 102 |
| Bob | 103 |
| Bob | 104 |
| Carol | 105 |
```

EXPLAIN ANALYZE 期望输出:

```
Project(columns=[c.name, o.order_id],  rows=5)
    Join(tables=[customers, orders],  condition=[c.customer_id=o.customer_id],
rows=5)
        Project(columns=[c.customer_id, c.name],  rows=3)
            Scan(table=customers,  type=SeqScan,  rows=3)
        Project(columns=[o.customer_id, o.order_id],  rows=15)
            Scan(table=orders,  type=SeqScan,  rows=15)
```

说明:

- customers 表为 NLJ 的左侧外表，单次扫描 3 行。
- orders 表作为右侧内表，被 customers 的 3 行重复扫描，共访问 `3 × 5 = 15` 行，因此右侧 `Project rows=15`、`Scan rows=15`。
- 投影下推后，customers 侧仅保留 `c.customer_id, c.name`，orders 侧仅保留 `o.customer_id, o.order_id`。
- `Join rows=5`：最终连接结果为 5 行。

题目五 聚合函数与分组统计

测试点1：单独使用聚合函数

测试示例:

```
create table grade (course char(20),id int,score float);

insert into grade values('DataStructure',1,95);

insert into grade values('DataStructure',2,93.5);

insert into grade values('DataStructure',4,87);

insert into grade values('DataStructure',3,85);

insert into grade values('DB',1,94);

insert into grade values('DB',2,74.5);

insert into grade values('DB',4,83);
```

```
insert into grade values('DB',3,87);

select MAX(id) as max_id from grade;

select MIN(score) as min_score from grade where course = 'DB';

select COUNT(course) as course_num from grade;

select COUNT(*) as row_num from grade;

select SUM(score) as sum_score from grade where id = 1;

drop table grade;
```

期待输出：

```
| max_id |
| 4 |
| min_score |
| 74.500000 |
| course_num |
| 8 |
| row_num |
| 8 |
| sum_score |
| 189.000000 |
```

测试点2：聚合函数加分组统计

测试示例:

```
create table grade (course char(20),id int,score float);

insert into grade values('DataStructure',1,95);

insert into grade values('DataStructure',2,93.5);

insert into grade values('DataStructure',3,94.5);

insert into grade values('ComputerNetworks',1,99);

insert into grade values('ComputerNetworks',2,88.5);

insert into grade values('ComputerNetworks',3,92.5);

insert into grade values('C++',1,92);

insert into grade values('C++',2,89);
```

```
insert into grade values('C++',3,89.5);

select id,MAX(score) as max_score,MIN(score) as min_score,SUM(score) as sum_score
from grade group by id;

select id,MAX(score) as max_score from grade group by id having COUNT(*) >  3;

insert into grade values ('ParallelCompute',1,100);

select id,MAX(score) as max_score from grade group by id having COUNT(*) >  3;

select id,MAX(score) as max_score,MIN(score) as min_score from grade group by id
having COUNT(*) > 1 and MIN(score) > 88;

select course ,COUNT(*) as row_num , COUNT(id) as student_num , MAX(score) as
top_score, MIN(score) as lowest_score from grade group by course;

drop table grade;
```

期待输出：

```
| id | max_score | min_score | sum_score |
| 1 |  99.000000 | 92.000000 | 286.000000 |
| 2 |  93.500000 | 88.500000 | 271.000000 |
| 3 |  94.500000 | 89.500000 | 276.500000 |
| id | max_score |
| 1 | 100.000000 |
| id | max_score | min_score |
| 1 | 100.000000 | 92.000000 |
| 2 | 93.500000 | 88.500000 |
| 3 | 94.500000 | 89.500000 |
| course | row_num | student_num | top_score | lowest_score |
| DataStructure | 3 | 3 | 95.000000 | 93.500000 |
| ComputerNetworks | 3 | 3 | 99.000000 | 88.500000 |
| C++ | 3 | 3 | 92.000000 | 89.000000 |
| ParallelCompute | 1 | 1 | 100.000000 | 100.000000 |
```

测试点3：健壮性测试

测试示例:

```
create table grade (course char(20),id int,score float);

insert into grade values('DataStructure',1,95);

insert into grade values('DataStructure',2,93.5);

insert into grade values('DataStructure',3,94.5);
```

```
insert into grade values('ComputerNetworks',1,99);

insert into grade values('ComputerNetworks',2,88.5);

insert into grade values('ComputerNetworks',3,92.5);

-- SELECT 列表中不能出现没有在 GROUP BY 子句中的非聚集列

select id , score from grade group by course;

-- WHERE 子句中不能用聚集函数作为条件表达式

select id, MAX(score) as max_score where MAX(score) > 90 from grade group by id;
```

期待输出：

```
failure
failure
```

测试点4：order by 语句测试

测试示例:

```
create table records (vendor char(5), invoice_number int, amount float);

insert into records values('alpha', 1001, 98.0);
insert into records values('bravo', 2002, 76.5);
insert into records values('charl', 3003, 99.0);
insert into records values('delta', 1001, 98.5);
insert into records values('echoo', 4004, 88.25);
insert into records values('foxxx', 4004, 77.0);
insert into records values('golfy', 5005, 97.75);
insert into records values('hotel', 5005, 86.75);
insert into records values('indio', 6006, 76.25);
insert into records values('julie', 3003, 88.0);
insert into records values('karen', 5005, 89.25);
insert into records values('lenny', 2002, 91.125);
insert into records values('mango', 6006, 98.5);
insert into records values('nancy', 1001, 89.75);
insert into records values('oscar', 2002, 90.0);
insert into records values('peter', 3003, 95.0);
insert into records values('quack', 6006, 88.625);
insert into records values('romeo', 4004, 92.0);
insert into records values('sunny', 1001, 95.25);
insert into records values('tonny', 7007, 98.125);
insert into records values('ultra', 4004, 91.5);
insert into records values('vivid', 7007, 98.3125);
```

```
select * from records order by invoice_number, amount asc limit 2;
```

期待输出：

```
| vendor | invoice_number | amount |
| nancy | 1001 | 89.750000 |
| sunny | 1001 | 95.250000 |
```

测试输出要求：

本题目的输出要求写入数据库文件夹下的`output.txt`文件中，例如测试数据库名称为`execution_test_db`，则在测试时使用`./bin/rmdb execution_test_db`命令来启动服务端，对应输出应写入`build/execution_test_db/output.txt`文件中。

题目六 Union 集合算子

测试点1：

```
CREATE TABLE orders1 (
    order_id INT,
    amount FLOAT,
    region CHAR(10)
);

CREATE TABLE orders2 (
    order_id INT,
    amount FLOAT,
    region CHAR(10)
);

CREATE TABLE orders3 (
    order_id INT,
    amount FLOAT,
    region CHAR(10)
);

INSERT INTO orders1 VALUES (1, 150.0, 'Beijing');
INSERT INTO orders1 VALUES (2, 230.5, 'Shanghai');
INSERT INTO orders1 VALUES (3, 89.99, 'Guangzhou');

INSERT INTO orders2 VALUES (1, 150.0, 'Beijing');
INSERT INTO orders2 VALUES (4, 120.0, 'Shenzhen');
INSERT INTO orders2 VALUES (5, 560.0, 'Chengdu');

INSERT INTO orders3 VALUES (1, 150.0, 'Beijing');
INSERT INTO orders3 VALUES (6, 199.99, 'Wuhan');

SELECT * FROM
```

```
    (SELECT * FROM orders1
     UNION
     SELECT * FROM orders2
     UNION
     SELECT * FROM orders3) AS all_orders
ORDER BY amount DESC;
```

预期结果：

```
| order_id | amount | region |
| 5 | 560 | Chengdu |
| 2 | 230.5 | Shanghai |
| 6 | 199.99 | Wuhan |
| 1 | 150 | Beijing |
| 4 | 120 | Shenzhen |
| 3 | 89.99 | Guangzhou |
```

测试点2:

```
CREATE TABLE orders1 (
    order_id INT,
    amount INT,
    region CHAR(10)
);

CREATE TABLE orders2 (
    order_id INT,
    amount FLOAT,
    region CHAR(20)
);

-- 错误示例 1：列数量不一致
SELECT * FROM
    (SELECT amount FROM orders1
     UNION
     SELECT * FROM orders2
    ) AS all_orders;

-- 错误示例 2：数据类型不兼容
SELECT * FROM
    (SELECT amount FROM orders1
     UNION
     SELECT region FROM orders2
    ) AS all_orders;

-- 错误示例 3：ORDER BY 出现未知列
SELECT * FROM
    (SELECT amount FROM orders1
     UNION
```

```
     SELECT amount FROM orders2
    ) AS all_orders
ORDER BY order_id ASC;

-- 类型提升测试示例
-- amount 列：INT 与 FLOAT 提升为 FLOAT
-- region 列：CHAR(10) 与 CHAR(20) 提升为 CHAR(20)
-- 预期结果的第一行（虚线部分）数据模式无需返回。
SELECT * FROM
    (SELECT * FROM orders1
     UNION
     SELECT * FROM orders2
    ) AS all_orders
ORDER BY amount DESC, order_id ASC;
```

错误示例预期结果：

- 错误示例 1 → `failure`
- 错误示例 2 → `failure`
- 错误示例 3 → `failure`

类型提升测试预期结果： 数据模式：`INT | FLOAT | CHAR(20)`

```
| order_id | amount | region |
| ... | ... | ... |
| ... | ... | ... |
| ... | ... | ... |
| ... | ... | ... |
| ... | ... | ... |
```

题目七 嵌套循环连接及其优化

测试点1: 双表连接示例

数据准备

```
CREATE TABLE departments (dept_id int, dept_name char(20));
CREATE TABLE employees (emp_id int, dept_id int, emp_name char(20), salary int);

INSERT INTO departments VALUES (10, 'Finance');
INSERT INTO departments VALUES (20, 'HR');
INSERT INTO departments VALUES (30, 'Sales');
INSERT INTO departments VALUES (40, 'Engineering');
INSERT INTO departments VALUES (50, 'Marketing');

INSERT INTO employees VALUES (1, 10, 'Alice', 65000);
INSERT INTO employees VALUES (2, 20, 'Bob', 48000);
INSERT INTO employees VALUES (3, 30, 'Charlie', 52000);
```

```
INSERT INTO employees VALUES (4, 40, 'Diana', 70000);
INSERT INTO employees VALUES (5, 50, 'Eve', 55000);
```

说明：

- `departments` 表（左表/驱动表）5 行，`dept_id` 为部门编号。
- `employees` 表（右表/被驱动表）5 行，每个员工属于唯一部门，`dept_id` 值各不相同。
- JOIN 结果为一一对应，共 5 行。

无索引：NLJ

执行如下命令，先确认结果正确性，再观察执行计划：

```
SELECT departments.dept_name, employees.emp_name FROM departments JOIN employees
ON departments.dept_id = employees.dept_id;

EXPLAIN ANALYZE SELECT departments.dept_name, employees.emp_name FROM departments
JOIN employees ON departments.dept_id = employees.dept_id;
```

SELECT 结果：

```
| dept_name | emp_name |
| Finance | Alice |
| HR | Bob |
| Sales | Charlie |
| Engineering | Diana |
| Marketing | Eve |
```

EXPLAIN ANALYZE 期望输出：

```
Project(columns=[departments.dept_name, employees.emp_name], rows=5)
    Join(tables=[departments, employees], condition=
[departments.dept_id=employees.dept_id], rows=5)
        Project(columns=[departments.dept_id, departments.dept_name], rows=5)
            Scan(table=departments, type=SeqScan, rows=5)
        Project(columns=[employees.dept_id, employees.emp_name], rows=25)
            Scan(table=employees, type=SeqScan, rows=25)
```

说明：

- `Join rows=5`：最终 Join 产出 5 行。
- 右侧 `Scan rows=25`：NLJ 中右表作为内表被左表 5 行重复全表扫描，共访问 `5 × 5 = 25` 行。
- 没有 `WHERE` 谓词时，不应输出 `Filter` 节点；连接条件应出现在 `Join(condition=[...])` 中。

有索引：INLJ

```
CREATE INDEX employees(dept_id);

SELECT departments.dept_name, employees.emp_name FROM departments JOIN employees
ON departments.dept_id = employees.dept_id;

EXPLAIN ANALYZE SELECT departments.dept_name, employees.emp_name FROM departments
JOIN employees ON departments.dept_id = employees.dept_id;
```

SELECT 结果（与 NLJ 结果相同）：

```
| dept_name | emp_name |
| Finance | Alice |
| HR | Bob |
| Sales | Charlie |
| Engineering | Diana |
| Marketing | Eve |
```

EXPLAIN ANALYZE 期望输出:

```
Project(columns=[departments.dept_name, employees.emp_name], rows=5)
    Join(tables=[departments, employees], condition=
[departments.dept_id=employees.dept_id], rows=5)
        Project(columns=[departments.dept_id, departments.dept_name], rows=5)
            Scan(table=departments, type=SeqScan, rows=5)
        Project(columns=[employees.dept_id, employees.emp_name], rows=5)
            Scan(table=employees, type=IndexScan, using_index=(dept_id), rows=5)
```

说明:

- `Scan type` 变为 `IndexScan`，且使用右表连接列 `dept_id` 上的索引。
- 右侧 `Scan rows=5`：由于使用了唯一索引点查（Lookup），内表仅根据连接键精确读取匹配行，无需全表扫描。
- 右表访问行数从 `25` 降为 `5`，性能提升显著。

测试点2: 多表连接示例

此处示例为三表连接查询。

数据准备

```
CREATE TABLE departments (dept_id int, dept_name char(20));
CREATE TABLE employees (emp_id int, dept_id int, emp_name char(20));
CREATE TABLE offices (office_id int, dept_id int, office_name char(20));

INSERT INTO departments VALUES (10, 'Finance');
```

```
INSERT INTO departments VALUES (20, 'HR');
INSERT INTO departments VALUES (30, 'Sales');

INSERT INTO employees VALUES (1, 10, 'Alice');
INSERT INTO employees VALUES (2, 20, 'Bob');
INSERT INTO employees VALUES (3, 30, 'Charlie');

INSERT INTO offices VALUES (101, 10, 'HQ');
INSERT INTO offices VALUES (102, 20, 'Remote');
INSERT INTO offices VALUES (103, 30, 'Field');
```

无索引：NLJ

执行如下命令：

```
SELECT departments.dept_name, employees.emp_name, offices.office_name
FROM departments
JOIN employees ON departments.dept_id = employees.dept_id
JOIN offices ON departments.dept_id = offices.dept_id;
```

SELECT 结果：

```
| dept_name | emp_name | office_name |
| Finance | Alice | HQ |
| HR | Bob | Remote |
| Sales | Charlie | Field |
```

无索引时的 `EXPLAIN ANALYZE` 结果如下：

```
Project(columns=[departments.dept_name, employees.emp_name, offices.office_name],
rows=3)
    Join(tables=[departments, employees, offices], condition=
[departments.dept_id=offices.dept_id], rows=3)
        Join(tables=[departments, employees], condition=
[departments.dept_id=employees.dept_id], rows=3)
            Project(columns=[departments.dept_id, departments.dept_name], rows=3)
                Scan(table=departments, type=SeqScan, rows=3)
            Project(columns=[employees.dept_id, employees.emp_name], rows=9)
                Scan(table=employees, type=SeqScan, rows=9)
        Project(columns=[offices.dept_id, offices.office_name], rows=9)
            Scan(table=offices, type=SeqScan, rows=9)
```

有索引：INLJ

先为每个 Join 层级的右侧内表连接列创建索引：

```
CREATE INDEX employees(dept_id);
CREATE INDEX offices(dept_id);

SELECT departments.dept_name, employees.emp_name, offices.office_name
FROM departments
JOIN employees ON departments.dept_id = employees.dept_id
JOIN offices ON departments.dept_id = offices.dept_id;

EXPLAIN ANALYZE SELECT departments.dept_name, employees.emp_name,
offices.office_name
FROM departments
JOIN employees ON departments.dept_id = employees.dept_id
JOIN offices ON departments.dept_id = offices.dept_id;
```

SELECT 结果（与无索引结果相同）：

```
| dept_name | emp_name | office_name |
| Finance | Alice | HQ |
| HR | Bob | Remote |
| Sales | Charlie | Field |
```

建索引后的 `EXPLAIN ANALYZE` 结果如下：

```
Project(columns=[departments.dept_name, employees.emp_name, offices.office_name],
rows=3)
    Join(tables=[departments, employees, offices], condition=
[departments.dept_id=offices.dept_id], rows=3)
        Join(tables=[departments, employees], condition=
[departments.dept_id=employees.dept_id], rows=3)
            Project(columns=[departments.dept_id, departments.dept_name], rows=3)
                Scan(table=departments, type=SeqScan, rows=3)
            Project(columns=[employees.dept_id, employees.emp_name], rows=3)
                Scan(table=employees, type=IndexScan, using_index=(dept_id),
rows=3)
        Project(columns=[offices.dept_id, offices.office_name], rows=3)
            Scan(table=offices, type=IndexScan, using_index=(dept_id), rows=3)
```

测试点3: 评测说明

`EXPLAIN ANALYZE SELECT ...` 只向输出文件写入执行计划与统计信息，不再额外写入 `SELECT` 查询结果。

评测系统将按照以下流程对选手的提交进行自动化测评：

1. **初始化**：创建表并导入基准测试数据集，此时不会创建任何辅助索引。
2. **NLJ 测试**：

- 执行标准查询并记录耗时 t1t1。检查结果集是否正确。

- 执行 `EXPLAIN ANALYZE`。检查执行计划是否符合逻辑。

3. **索引构建**：在内表连接字段上执行 `CREATE INDEX table_name(column_name);`。
4. **INLJ 测试**：

- 再次执行标准查询并记录耗时 t2t2。检查结果集是否依然正确。
- 执行 `EXPLAIN ANALYZE`。检查执行计划是否符合逻辑。

5. **性能记录**：对大数据测试点记录加速比，用于检测 INLJ 是否确实带来性能收益。

题目八 事务控制语句

测试点

测试示例：

```
create table student (id int, name char(8), score float);
insert into student values (1, 'xiaohong', 90.0);
begin;
insert into student values (2, 'xiaoming', 99.0);
delete from student where id = 2;
abort;
select * from student;
```

期待输出：

```
| id | name | score |
| 1 | xiaohong | 90.000000 |
```

测试文件说明

- commit_test：事务提交测试，不包含索引
- abort_test：事务回滚测试，不包含索引
- commit_index_test：事务提交测试，包含索引
- abort_index_test：事务回滚测试，包含索引

题目九 可配置的快照隔离与可串行化隔离级别

下面给出若干参考示例帮助理解本题的判分方式。示例中的 SQL 按逻辑划分为 `T1`、`T2`、`T3` 三个事务，实际测试中可能由多个会话交错执行。

示例一：基本并发样例，同一记录上的写写冲突

该样例用于验证对同一条记录的并发写冲突处理。该样例在 `SI` 和 `SER` 下结果一致。

初始化 SQL：

```
create table account (id int, balance int);
insert into account values (1, 100);
```

事务 1：

```
t1s set transaction isolation level <LEVEL>;
t1a begin;
t1b update account set balance = 120 where id = 1;
t1c commit;
```

事务 2：

```
t2s set transaction isolation level <LEVEL>;
t2a begin;
t2b update account set balance = 90 where id = 1;
t2c commit;
```

事务 3：

```
t3a select * from account where id = 1;
```

调度顺序如下：

步骤	T1	T2	T3
1	t1s set transaction isolation level <LEVEL>;		
2		t2s set transaction isolation level <LEVEL>;	
3	t1a begin;		
4	t1b update account set balance = 120 where id = 1;		
5		t2a begin;	
6		t2b update account set balance = 90 where id = 1;	
7	t1c commit;		
8		t2c commit;	

步骤	T1	T2	T3
9			t3a select * from account where id = 1;

期望标准输出：

```
abort
+------------------+------------------+
|               id |          balance |
+------------------+------------------+
|                1 |              120 |
+------------------+------------------+
Total record(s): 1
```

样例说明：

1. T2 不能在 T1 未结束时成功覆盖 T1 对同一记录的更新。
2. T2 应在冲突写语句 t2b 返回 abort，并回滚事务 2。
3. 若 T2 已经在 t2b 被回滚，则后续 t2c 不应使事务 2 的写入生效。
4. T3 的查询结果应为 (1, 120)。

示例二：事务级快照的一致性

该样例用于验证事务级快照语义。在 SI 与 SER 下，事务读取的都是其开始时的 MVCC 快照，再叠加本事务自身写入。

初始化 SQL：

```
create table counter_test (id int, val int);
insert into counter_test values (1, 100);
```

事务 1：

```
t1s set transaction isolation level <LEVEL>;
t1a begin;
t1b select * from counter_test where id = 1;
t1c select * from counter_test where id = 1;
t1d commit;
```

事务 2：

```
t2a begin;
t2b update counter_test set val = 200 where id = 1;
t2c commit;
```

调度顺序如下：

步骤	T1	T2
1	t1s set transaction isolation level <LEVEL>;	
2	t1a begin;	
3	t1b select * from counter_test where id = 1;	
4		t2a begin;
5		t2b update counter_test set val = 200 where id = 1;
6		t2c commit;
7	t1c select * from counter_test where id = 1;	
8	t1d commit;	

当 <LEVEL> = SNAPSHOT ISOLATION 时，期望标准输出为：

```
+------------------+------------------+
|               id |              val |
+------------------+------------------+
|                1 |              100 |
+------------------+------------------+
Total record(s): 1
+------------------+------------------+
|               id |              val |
+------------------+------------------+
|                1 |              100 |
+------------------+------------------+
Total record(s): 1
```

样例说明：

1. 两种隔离级别都不允许脏读。
2. 在同一事务中，即使并发事务已经提交新的写入结果，T1 的第二次查询仍应保持与第一次查询一致。
3. 该样例主要验证 SI 必须满足事务级快照的一致性。

示例三：写偏序场景

写偏序是用于区分 `SNAPSHOT ISOLATION` 与 `SERIALIZABLE` 隔离级别的关键场景之一。

场景描述：

表 `duty` 记录两位值班医生是否在岗，约束为“任意时刻至少有一位医生在岗”。初始时两位医生都在岗。两个事务分别读取“另一位医生仍在岗”的事实后，都将自己改为离岗。该调度会形成相反方向的读写反依赖，是典型写偏序异常。

初始化 SQL：

```
create table duty (doctor_id int, on_call int);
insert into duty values (1, 1);
insert into duty values (2, 1);
```

事务 1：

```
t1s set transaction isolation level <LEVEL>;
t1a begin;
t1b select * from duty where doctor_id = 2;
t1c update duty set on_call = 0 where doctor_id = 1;
t1d commit;
```

事务 2：

```
t2s set transaction isolation level <LEVEL>;
t2a begin;
t2b select * from duty where doctor_id = 1;
t2c update duty set on_call = 0 where doctor_id = 2;
t2d commit;
```

事务 3：

```
t3a select * from duty;
```

调度顺序如下：

步骤	T1	T2	T3
1	`t1s set transaction isolation level <LEVEL>;`		

步骤	T1	T2	T3
2		t2s set transaction isolation level <LEVEL>;	
3	t1a begin;		
4		t2a begin;	
5	t1b select * from duty where doctor_id = 2;		
6		t2b select * from duty where doctor_id = 1;	
7	t1c update duty set on_call = 0 where doctor_id = 1;		
8		t2c update duty set on_call = 0 where doctor_id = 2;	
9	t1d commit;		
10		t2d commit;	
11			t3a select * from duty;

当 `<LEVEL> = SNAPSHOT ISOLATION` 时，期望标准输出为：

```
+------------------+------------------+
|        doctor_id |          on_call |
+------------------+------------------+
|                2 |                1 |
+------------------+------------------+
Total record(s): 1
+------------------+------------------+
|        doctor_id |          on_call |
+------------------+------------------+
|                1 |                1 |
+------------------+------------------+
Total record(s): 1
+------------------+------------------+
|        doctor_id |          on_call |
+------------------+------------------+
|                1 |                0 |
|                2 |                0 |
+------------------+------------------+
Total record(s): 2
```

此时 T1 和 T2 均可成功提交。T1 与 T2 读取的是各自事务开始时的快照，因此都能看到"另一位医生仍在岗"；由于两个事务更新的是不同记录，不发生同一记录上的写写冲突，最终可能出现两人都离岗的写偏序结果。

当 <LEVEL> = SERIALIZABLE 时，期望标准输出为：

```
+------------------+------------------+
|        doctor_id |          on_call |
+------------------+------------------+
|                2 |                1 |
+------------------+------------------+
Total record(s): 1
+------------------+------------------+
|        doctor_id |          on_call |
+------------------+------------------+
|                1 |                1 |
+------------------+------------------+
Total record(s): 1
abort
+------------------+------------------+
|        doctor_id |          on_call |
+------------------+------------------+
|                1 |                0 |
|                2 |                1 |
+------------------+------------------+
Total record(s): 2
```

在 SER 下，t1c 先建立 T2 ->rw T1，尚未形成危险结构；t2c 再建立 T1 ->rw T2，此时形成 SSI 危险结构。根据本题固定的当前语句回滚规则，t2c 需要返回 abort，T2 被回滚，最终只有 T1的更新生效。

测试点说明

测试点包括但不限于以下内容：

测试点	详细说明
syntax	配置隔离级别的语法正确性测试
si/InsertTest	在 SI 下验证插入记录的事务内可见性、并发事务快照可见性以及提交后的最终查询结果
si/DirtyReadTest	在 SI 下验证未提交更新不会被其他事务读到，同时事务能够读到自己的未提交写入
si/ReadWriteConflictDeleteTest	在 SI 下验证读后并发删除、再删除同一记录时的快照可见性和删除冲突处理
si/WriteWriteConflictUpdateTest	在 SI 下验证两个事务并发更新同一记录时的写写冲突处理，最终只能保留一个有效写入结果
ser/UnrepeatableReadTest	在 SER 下验证重复读取同一记录时的快照读结果和 SSI 依赖判断
ser/PhantomReadTest	在 SER 下验证范围读、空结果读与并发写入之间的谓词读依赖判断

测试点	详细说明
ser/WriteSkewTest	在 SER 下执行写，由于写语句触发 SSI 危险结构，验证系统能够回滚导致 SSI 危险结构成立的事务并维持全局约束
ser/SelectDangerousTest	在 SER 下执行，由读语句触发 SSI 危险结构，验证系统能够回滚导致 SSI 危险结构成立的事务并维持全局约束

测试输出要求

1. 对于写写冲突，冲突写语句需要向客户端返回 abort\n，并回滚执行该写语句的事务。
2. 对于 SSI 危险结构，新增 rw 反依赖并导致危险结构成立的当前语句需要向客户端返回 abort\n，并回滚当前语句所在事务；当前语句可以是 SELECT、INSERT、UPDATE 或 DELETE。
3. 除 select 的标准输出以及必要的事务回滚信息之外，不应向客户端返回多余信息。
4. 服务端的输出仍应写入对应数据库目录下的 output.txt 文件中。

题目十 基于静态检查点的故障恢复-测试方案

测试点说明

本题目包含6个测试点，其中，前5个测试点是不包括静态检查点的故障恢复，第6个测试点是包含静态检查点的故障恢复：

测试点	详细说明
crash_recovery_single_thread_test	单线程发送事务，数据量较小，不包括建立检查点
crash_recovery_multi_thread_test	多线程发送事务，数据量较小，不包括建立检查点
crash_recovery_index_test	单线程发送事务，包含建立索引，数据量较大，不包括建立检查点
crash_recovery_large_data_test	多线程发送事务，数据量较大，不包括建立检查点
crash_recovery_without_checkpoint	单线程发送事务，数据量巨大，不包括建立检查点，会记录系统故障恢复时间t1（注：恢复时间指从server重启开始，到server能够提供服务为止所需的时间）
crash_recovery_with_checkpoint	单线程发送事务，数据量同crash_recovery_without_checkpoint测试，在执行过程中，会不定时发送create static_checkpoint建立检查点，在系统crash之后会记录系统恢复时间t2，其中t2不超过t1的70%并且通过本题目的一致性检测，可认为通过本测试点

测试过程说明

2.1 创建表，并插入一定量的数据

```
create table warehouse (w_id int, w_name char(10), w_street_1 char(20), w_street_2
char(20), w_city char(20), w_state char(2), w_zip char(9), w_tax float, w_ytd
float);
create table district (d_id int, d_w_id int, d_name char(10), d_street_1 char(20),
d_street_2 char(20), d_city char(20), d_state char(2), d_zip char(9), d_tax float,
```

```
d_ytd float, d_next_o_id int);
create table customer (c_id int, c_d_id int, c_w_id int, c_first char(16),
c_middle char(2), c_last char(16), c_street_1 char(20), c_street_2 char(20),
c_city char(20), c_state char(2), c_zip char(9), c_phone char(16), c_since
char(30), c_credit char(2), c_credit_lim int, c_discount float, c_balance float,
c_ytd_payment float, c_payment_cnt int, c_delivery_cnt int, c_data char(50));
create table history (h_c_id int, h_c_d_id int, h_c_w_id int, h_d_id int, h_w_id
int, h_date char(19), h_amount float, h_data char(24));
create table new_orders (no_o_id int, no_d_id int, no_w_id int);
create table orders (o_id int, o_d_id int, o_w_id int, o_c_id int, o_entry_d
char(19), o_carrier_id int, o_ol_cnt int, o_all_local int);
create table order_line ( ol_o_id int, ol_d_id int, ol_w_id int, ol_number int,
ol_i_id int, ol_supply_w_id int, ol_delivery_d char(30), ol_quantity int,
ol_amount float, ol_dist_info char(24));
create table item (i_id int, i_im_id int, i_name char(24), i_price float, i_data
char(50));
create table stock (s_i_id int, s_w_id int, s_quantity int, s_dist_01 char(24),
s_dist_02 char(24), s_dist_03 char(24), s_dist_04 char(24), s_dist_05 char(24),
s_dist_06 char(24), s_dist_07 char(24), s_dist_08 char(24), s_dist_09 char(24),
s_dist_10 char(24), s_ytd float, s_order_cnt int, s_remote_cnt int, s_data
char(50));
insert ...
insert ...
insert ...
```

2.2 执行事务

```
begin;
select c_discount, c_last, c_credit, w_tax from customer, warehouse where w_id=1
and c_w_id=w_id and c_d_id=1 and c_id=2;
select d_next_o_id, d_tax from district where d_id=1 and d_w_id=1;
update district set d_next_o_id=5 where d_id=1 and d_w_id=1;
insert into orders values (4, 1, 1, 2, '2023-06-03 19:25:47', 26, 5, 1);
insert into new_orders values (4, 1, 1);
select i_price, i_name, i_data from item where i_id=10;
select s_quantity, s_data, s_dist_01, s_dist_02, s_dist_03, s_dist_04, s_dist_05,
s_dist_06, s_dist_07, s_dist_08, s_dist_09, s_dist_10 from stock where s_i_id=10
and s_w_id=1;
update stock set s_quantity=7 where s_i_id=10 and s_w_id=1;
insert into order_line values (4, 1, 1, 1, 10, 1, '2023-06-03 19:25:47', 7,
286.625000, 'VF2uQHlDhtxa5dKhPwWyCqgY');
select i_price, i_name, i_data from item where i_id=10;
```

测试点6会随机创建checkpoint

```
create static_checkpoint;
```

2.3 发生crash

```
crash
```

2.4 重启server

```
./bin/rmdb checkpoint_test
```

2.5 由另外一个线程一直尝试重连，统计恢复时间（仅最后两个测试点需要）

```
std::string query = "select * from district;";
time_t start_time = now();
while(true) {
    ret = connect_database("rmdb"); // 尝试重连
    if(ret == 0) {
        if(run_query(query)) {      // 如果重连成功并且执行事务成功，视为系统已经恢复完成
            break;
        }
    }
    sleep(0.05);                    // 否则等待0.05s之后再次尝试
}

time_t end_time = now()
time_t recovery_time = end_time - start_time    // 统计恢复时间
```

2.6 一致性检测（详见一致性检测文档），前五个测试点通过一致性测试即可拿到分数，最后一个测试点除了通过一致性测试之外，还需要故障恢复时间t2小于测试点5的故障恢复时间t1的70%